

Decentralized Gossip-SDCA: The Weight-Averaging Problem and Its Fix

This note explains, mathematically, a convergence bug observed when running this project's SVM training as a decentralized **gossip** protocol (both the PeerSim-Python engine in `src/network_layer/peersim_python/` and the p2pfl engine in `src/network_layer/own_network/`). The local solver is **SDCA** (Stochastic Dual Coordinate Ascent). When each node naively **averages its weight vector** with its neighbours, training does not converge — the duality gap and hinge loss oscillate with growing amplitude. This document explains why that happens and what the correct method is (**CoCoA / CoLA**: communicate weight *increments*, not absolute weights).

1. Problem setup and notation

We train an L2-regularized, hinge-loss linear SVM. There are N training examples (x_i, y_i) with $x_i \in \mathbb{R}^d$ and labels $y_i \in \{-1, +1\}$, and a regularization parameter $\lambda > 0$. The **global** objective is:

$$P(w) = \frac{1}{N} \sum_{i=1}^N \max(0, 1 - y_i w^T x_i) + \frac{\lambda}{2} \|w\|^2$$

In the decentralized setting there are K nodes (workers). The index set $1, \dots, N$ is partitioned into disjoint blocks $P_1, \dots, P_K$, and node k physically holds only its own examples (x_i, y_i) : $i \in P_k$. No node sees another node's raw data. The goal of gossip learning is that **every node ends up with the same model w** — the one it would have obtained by training on all N examples at once — with **no central server** and **no single point of failure**.

2. SDCA on a single machine

SDCA does not optimize $P(w)$ directly. It optimizes the **dual** problem. Using box variables $a_i \in [0, 1]$, the dual objective and the primal-dual link are:

$$D(a) = \frac{1}{N} \sum_{i=1}^N a_i - \frac{\lambda}{2} \|w(a)\|^2, \quad a_i \in [0, 1]$$

$$w(a) = \frac{1}{\lambda N} \sum_{i=1}^N a_i y_i x_i$$

The second equation is the crucial one: **the weight w is not a free parameter — it is a deterministic function of the dual variables a** . The a_i are the true state of the algorithm;

w is just their running total.

2.1 The closed-form coordinate update

SDCA maximizes D one coordinate at a time. Because $w(a)$ is affine in a_i and $\|w\|^2$ is quadratic, D restricted to a single a_i is a concave quadratic, maximized in closed form and clipped to $[0,1]$:

$$a_i^{\text{new}} = \Pi_{[0,1]} \left(a_i + \frac{1 - y_i x_i^T w}{\|x_i\|^2 / (\lambda N)} \right)$$

Here $\Pi_{[0,1]}$ clips to the box. After updating a_i , the weight is kept consistent by an incremental correction:

$$\Delta a_i = a_i^{\text{new}} - a_i, \quad w \leftarrow w + \frac{\Delta a_i y_i}{\lambda N} x_i$$

This is exactly the update implemented in `src/model.py` and in the `PeerSim` `SDCAProtocol._local_epoch`. No QP solver, no gradient step — one closed-form line.

2.2 The invariant that makes it work

Because the code corrects w every time any a_i changes, the following identity holds at **every** step. Call it invariant **(I)**:

$$w = \frac{1}{\lambda N} \sum_{i=1}^N a_i y_i x_i$$

The closed-form update in §2.1 is *derived under (I)*. Look at the two occurrences of the state in that formula: the term $y_i x_i^T w$ uses w, while the leading a_i term uses the dual variable directly. The derivation assumes these are two views of the same thing — that w already contains a_i 's contribution at full weight. If (I) is broken, the two views disagree and the "closed-form optimum" is computed at an inconsistent point.

2.3 Why single-machine SDCA converges

Under (I), each coordinate step *exactly maximizes* D along that coordinate, so the dual objective never decreases:

$$D(a^{t+1}) \geq D(a^t)$$

By weak duality $P(w(a)) \geq D(a)$ always, so the non-negative **duality gap** $P(w(a)) - D(a)$ is squeezed to zero. That monotone dual ascent is the entire convergence engine — and it depends on (I) holding.

3. The decentralized decomposition

Split the sum in the primal-dual link across the partition. By linearity:

$$w(a) = \sum_{k=1}^K w_k, \quad w_k = \frac{1}{\lambda N} \sum_{i \in P_k} a_i y_i x_i$$

This is the structural fact the whole fix rests on: **the global weight is the SUM of each node's partial weight w_k , not the average.** Each node owns a block of dual variables; its block produces a partial weight w_k ; the true global model is those partials **added together**.

4. The problem: naive weight averaging

The buggy scheme (what "just average the weights" does) is: each node runs a local SDCA epoch, then replaces its weight with an (age-weighted) average of its own and a neighbour's weight:

$$W_k \leftarrow \frac{t_k w_k + t_j w_j}{t_k + t_j}$$

This breaks the algorithm in three linked ways.

(a) It averages quantities that are meant to add. From §3 the pieces should combine as $\sum_k w_k$. Averaging them mis-scales: for two nodes, $(w_k + w_j)/2$ is only *half* the intended combined contribution. More importantly, after the average node k 's weight is no longer equal to $(1)/(\lambda N) \sum_{i \in P_k} a_i y_i x_i$ — **invariant (I) is violated locally**.

(b) The next SDCA step is computed at an inconsistent point. Write the averaged weight as $w_{\text{avg}} = c w_k + (\text{foreign part})$ with $c < 1$ (e.g. $c = 1/2$). Node k 's own dual contribution now sits inside w at scale c , but the leading a_i term in the update formula still uses a_i at scale 1. The "remove my own contribution, optimize, add it back" bookkeeping that the closed form performs no longer balances, so a_i^{new} overshoots the true coordinate optimum. The step maximizes *no* consistent objective.

(c) The ascent guarantee is gone. Since the step no longer maximizes any coordinate of a single D , the monotonicity $D(a^{t+1}) \geq D(a^t)$ fails. The duality gap — now computed with a primal w (averaged) and a dual a (local) that are out of sync — is not even a valid optimization gap; it can and does grow.

4.1 What this looks like empirically

Running the PeerSim gossip-SDCA on covtype for 100 cycles with naive averaging, the best model appears in the first ~15 cycles, then every gossip round makes it worse. The duality gap starts near 0.034, dips to ≈ 0.004 , then climbs in a growing saw-tooth to ≈ 0.18 ; the hinge loss rises from ≈ 0.58 to ≈ 0.80 . That saw-tooth (down, up, down, up — each peak higher) is the signature of a non-descent update, not of slow convergence.

5. The fix: CoCoA / CoLA — communicate increments, not weights

The correct family of methods is **CoCoA** (Communication-efficient distributed dual Coordinate Ascent) and its fully-decentralized form **CoLA**. The core idea follows directly from §3: keep each node's dual block private, keep a shared copy of the global w , and gossip the **change** each node makes — never the absolute weight.

5.1 The per-round algorithm

Every node keeps its own block a_{p_k} (never transmitted) and a copy of the shared global weight w . Each round, starting from the agreed weight w_{old} :

1. Node k initializes a local working copy $w_{\text{loc}} \leftarrow w_{\text{old}}$.
2. It runs one SDCA epoch over **its own** examples P_k , using w_{loc} in the closed-form update of §2.1 and keeping w_{loc} consistent with its block via §2.1's incremental correction. This produces a proposed block change Δa_{p_k} and a net local increment:

$$\Delta w_k = \frac{1}{\lambda N} \sum_{i \in P_k} \Delta a_i y_i x_i = w_{\text{loc}} - w_{\text{old}}$$

3. Nodes exchange the increments Δw_k (this is the gossip message) and apply the aggregate with a scaling $\gamma \in (0,1]$, applying the **same** scaling to the dual block and the weight:

$$a_{p_k} \leftarrow a_{p_k} + \gamma \Delta a_{p_k}, \quad w \leftarrow w + \gamma \sum_{k=1}^K \Delta w_k$$

The safe default is averaging, $\gamma = 1/K$; the more aggressive CoCoA+ variant uses $\gamma = 1$ with the local subproblem scaled correspondingly.

5.2 Why this preserves the invariant (and therefore converges)

Applying the same scaling γ to both the dual update and the weight update keeps invariant (I) intact network-wide: after the round, $w = (1)/(\lambda N) \sum_i a_i y_i x_i = \sum_k w_k$ still holds, because the weight moved by exactly the (scaled) sum of the increments produced *from* the dual changes. Every node's w therefore stays the true image of the combined dual variables. Because the local subproblems are genuine block ascent steps on the shared D , the aggregate is a valid ascent step, and (with γ and the subproblem scaling σ' satisfying $\sigma' \geq \gamma K$) the CoCoA+ theory guarantees the duality gap contracts:

$$0 \leq P(w(a)) - D(a) \leq \varepsilon \quad \text{after } O(1/\varepsilon) \text{ rounds (general convex / hinge).}$$

Contrast the two update rules directly. Broken — average the absolute weights:

$$w \leftarrow \frac{1}{K} \sum_{k=1}^K w_k$$

Fixed — add the scaled sum of increments on top of the retained weight:

$$w \leftarrow w + \gamma \sum_{k=1}^K \Delta w_k$$

You still average — but you average the **increments** and add them on top of the retained global weight, instead of averaging the absolute weights and discarding w_{old} .

5.3 The decentralized (gossip) version: CoLA

The exact sum $\sum_k \Delta w_k$ in §5.1 is an all-reduce. On a sparse gossip graph there is no all-reduce; CoLA replaces it with **iterative neighbour mixing**: each node repeatedly averages its increment estimate with its neighbours, and over rounds this consensus step converges to the network aggregate — recovering the CoCoA update with **no central node**. CoLA proves convergence over arbitrary connected communication graphs and is robust to changing topology. This maps directly onto the repo's gossip transport (per-node inbox + neighbour exchange).

5.4 Fault tolerance is preserved

Everything the gossip design is meant to give survives the fix:

- Every node holds a full copy of w that already reflects all contributions, so it is a redundant replica of the global model.
- If a node dies, its Δw_k is simply absent from that round's aggregate; the survivors continue and still hold a valid global model.
- There is no coordinator — the aggregate is formed by neighbours exchanging increments. No single point of failure.

The only thing that changed versus the broken scheme is the **payload** (Δw instead of w) and the **scaling** (γ on both dual and weight, with the global N). The decentralization, redundancy, and failure semantics are identical.

6. Summary: broken vs. fixed

- **Broken** — gossip the absolute weight and average it: $w \leftarrow \text{average}(w_k, w_j)$. Breaks invariant (I), desyncs w from the local a , loses the ascent guarantee, diverges.
- **Fixed (CoCoA/CoLA)** — gossip the round's increment and add the scaled sum: $w \leftarrow w + \gamma \sum_k \Delta w_k$, with $a_p \leftarrow a_p + \gamma \Delta a_p$. Preserves (I), keeps w consistent with a , is a valid block ascent step, converges — while remaining fully decentralized and fault-tolerant.

7. Implementation changes in this repository

The fix is contained entirely inside the local learner (SDCAProtocol for the PeerSim engine; the analogous SDCA module + aggregator for p2pfl):

- **Use the global N** in the update denominator (λN), not the local sample count n_k , so the per-node partials compose into the correct global weight.
- **Track the per-round increment** $\Delta w_k = w_{\text{loc}} - w_{\text{old}}$.
- **Change the gossip payload** from the absolute weight w to the increment Δw_k .
- **Change the merge** from an age-weighted average of absolute weights to $w \leftarrow w + \gamma \sum \Delta w$ (with $\gamma = 1/K$), applying the same γ to the dual block update.
- The **duality-gap metric** becomes well-defined again, because w and a are once more consistent.

Topology, the inbox/neighbour gossip transport, and the convergence-threshold observer are unchanged.

8. References

1. S. Shalev-Shwartz and T. Zhang. *Stochastic Dual Coordinate Ascent Methods for Regularized Loss Minimization*. JMLR, 2013.
2. M. Jaggi, V. Smith, M. Takáč, J. Terhorst, S. Krishnan, T. Hofmann, M. I. Jordan. *Communication-Efficient Distributed Dual Coordinate Ascent (CoCoA)*. NeurIPS, 2014.
3. C. Ma, V. Smith, M. Jaggi, M. I. Jordan, P. Richtárik, M. Takáč. *Adding vs. Averaging in Distributed Primal-Dual Optimization (CoCoA+)*. ICML, 2015.
4. L. He, A. Bian, M. Jaggi. *COLA: Decentralized Linear Learning*. NeurIPS, 2018.
5. I. Hegedűs, G. Danner, M. Jelasity. *Gossip Learning as a Decentralized Alternative to Federated Learning*. DAIS 2019 / Future Generation Computer Systems, 2021.